

Experiment 8: Pivot Table – Item Wise Unit Sold

Aim:

To create a Pivot Table using Pandas and find the total units sold for each item.

Procedure:

1. Import the Pandas library.
2. Read the sales_data.csv file using read_csv().
3. Create a Pivot Table with Item as the index and Units as the values.
4. Use the sum aggregation function to calculate total units sold.
5. Display the Pivot Table.

Program

```
import pandas as pd

# Read CSV file
df = pd.read_csv("sales_data.csv")

# 8. Item wise unit sold
print("Item Wise Unit Sold")
pivot1 = pd.pivot_table(df, values="Units", index="Item", aggfunc="sum")
print(pivot1)

# 9. Total sale amount region wise
print("\nRegion Wise Total Sale Amount")
pivot2 = pd.pivot_table(df, values="Sale_amt", index="Region", aggfunc="sum")
print(pivot2)

# Manager wise
print("\nManager Wise Total Sale Amount")
pivot3 = pd.pivot_table(df, values="Sale_amt", index="Manager", aggfunc="sum")
print(pivot3)

# Salesman wise
print("\nSalesMan Wise Total Sale Amount")
pivot4 = pd.pivot_table(df, values="Sale_amt", index="SalesMan", aggfunc="sum")
print(pivot4)
```

OUTPUT

Item Wise Unit Sold	
Item	Units
Cell Phone	91
Desk	2
Home Theater	308
Television	509
Video Games	16
Region Wise Total Sale Amount	
Region	Sale_amt
Central	393943
East	286076
West	105424
Manager Wise Total Sale Amount	
Manager	Sale_amt
Douglas	79086
Hermann	297768
Martha	335426
Timothy	73163
SalesMan Wise Total Sale Amount	
SalesMan	Sale_amt
Alexander	231076
David	6075
Diana	14500
John	250
Karen	40500
Luis	150948
Michael	38336
Shelli	25000
Sigal	121820
Stephen	67088
Steven	89850

Experiment 9: Pivot Table – Total Sale Amount Region Wise, Manager Wise, and SalesMan Wise

Aim

To create Pivot Tables using Pandas and find the total sale amount region-wise, manager-wise, and salesman-wise from the given sales data.

Procedure

1. Import the Pandas library.
2. Read the `sales_data.csv` file using `pd.read_csv()`.
3. Create a Pivot Table with Region as the index and Sale_amt as the values using the sum aggregation function.
4. Create another Pivot Table with Manager as the index and Sale_amt as the values using the sum function.
5. Create a third Pivot Table with SalesMan as the index and Sale_amt as the values using the sum function.
6. Display all the Pivot Tables

Program

```
import pandas as pd

# Read the CSV file
df = pd.read_csv("sales_data.csv")

# Region-wise total sale amount
print("Region Wise Total Sale Amount")
region = pd.pivot_table(df, values="Sale_amt", index="Region", aggfunc="sum")
print(region)

# Manager-wise total sale amount
print("\nManager Wise Total Sale Amount")
manager = pd.pivot_table(df, values="Sale_amt", index="Manager", aggfunc="sum")
print(manager)

# SalesMan-wise total sale amount
print("\nSalesMan Wise Total Sale Amount")
salesman = pd.pivot_table(df, values="Sale_amt", index="SalesMan", aggfunc="sum")
print(salesman)
```

OUTPUT

Region Wise Total Sale Amount	
	Sale_amt
Region	
Central	393943
East	286076
West	105424
Manager Wise Total Sale Amount	
	Sale_amt
Manager	
Douglas	79086
Hermann	297768
Martha	335426
Timothy	73163
SalesMan Wise Total Sale Amount	
	Sale_amt
SalesMan	
Alexander	231076
David	6075
Diana	14500
John	250
Karen	40500
Luis	150948
Michael	38336
Shelli	25000
Sigal	121820
Stephen	67088
Steven	89850

Experiment 10: Highlighting Negative Numbers in Red and Positive Numbers in Black Using Pandas DataFrame

Aim

To create a DataFrame with 10 rows and 5 columns containing random values and highlight negative numbers in red and positive numbers in black using Pandas.

Procedure

1. Import the Pandas and NumPy libraries.
2. Create a DataFrame with random values using `numpy.random.randn()`.
3. Define a function to assign red color to negative values and black color to positive values.
4. Apply the function using the `style.applymap()` method.
5. Display the styled DataFrame.

PROGRAM

```
import pandas as pd
import numpy as np

# Create DataFrame
np.random.seed(100)
df = pd.DataFrame(np.random.randn(10,5),
                  columns=['A', 'B', 'C', 'D', 'E'])

# Function to highlight values
def color_negative_red(value):
    if value < 0:
        return 'color:red'
    else:
        return 'color:black'

# Display styled DataFrame
print("Original DataFrame:")
print(df)

# Highlight negative values in red and positive values in black
styled_df = df.style.applymap(color_negative_red)

# Save the styled DataFrame as an HTML file
styled_df.to_html("output.html")

print("\nStyled DataFrame saved as 'output.html'")
print("Open output.html in your web browser to see the colors.")
```

OUTPUT

	A	B	C	D	E
0	-1.749765	0.342680	1.153036	-0.252436	0.981321
1	0.514219	0.221180	-1.070043	-0.189496	0.255001
2	-0.458027	0.435163	-0.583595	0.816847	0.672721
3	-0.104411	-0.531280	1.029733	-0.438136	-1.118318
4	1.618982	1.541605	-0.251879	-0.842436	0.184519
5	0.937082	0.731000	1.361556	-0.326238	0.055676
6	0.222400	-1.443217	-0.756352	0.816454	0.750445
7	-0.455947	1.189622	-1.690617	-1.356399	-1.232435
8	-0.544439	-0.668172	0.007315	-0.612939	1.299748
9	-1.733096	-0.983310	0.357508	-1.613579	1.470714